Lab 6: PyTorch and Actor-Critic Methods

Morgan Cooper
MSDS 684 — Reinforcement Learning

April 25, 2026

Section 1: Project Overview

This lab moves from value-based methods to policy-based methods. Instead of learning Q-values and deriving a policy from them like in Lab 5, the actor-critic algorithm learns the policy directly with a neural network. A second neural network, the critic, learns the state value $V(s)$ and tells the actor how good a state is. The actor uses this signal as an advantage to update itself with less variance than REINFORCE alone. PyTorch is used here for the first time so that gradients are computed automatically instead of by hand.

Pendulum-v1 has a continuous action space, so a softmax policy like in CartPole will not work. Instead the actor outputs the mean of a Gaussian distribution and learns the standard deviation as a parameter. Actions are sampled from this distribution and clipped to the action range before stepping the environment. The critic uses the same kind of value function approximator as Lab 5, but now it is a neural network instead of tile coding, and it is updated with TD(0) bootstrapping every step.

Pendulum-v1 (Gymnasium):

- State space: Box([-1, -1, -8], [1, 1, 8]) — $\cos\theta$, $\sin\theta$, and angular velocity
- Action space: 1 continuous action — torque between -2 and $+2$
- Rewards: $-(\theta^2 + 0.1\,\dot{\theta}^2 + 0.001\,a^2)$ per step, best near 0 when upright and still
- Terminal condition: 200 step time limit, no early termination

I hypothesize that training will be unstable at first because actor-critic with TD(0) is known to have high variance. From earlier labs I expect to need several debugging cycles to fix issues like the actor mean saturating outside the action range or the standard deviation drifting too high. Once those are fixed, I expect the agent to learn to swing the pendulum upright and hold it there. Running 30 random seeds should show how reliable the method is across different initializations.

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/policy-gradient-rl-methods`

Implementation Summary

I implemented online actor-critic with TD(0) on Pendulum-v1 using PyTorch. The actor is a neural network with two hidden layers of 64 tanh units. It outputs the mean of a Gaussian policy. The standard deviation is a separate learned parameter clamped so sigma stays between 0.13 and 0.50. The critic uses the same hidden layers but outputs a single state value $V(s)$. Both networks use Adam with the actor at 3e-4 and the critic at 1e-3. I set gamma=0.99, entropy_coef=0.01, and grad_clip=1.0. Furthermore, I added several fixes during debugging including a tanh squash on the actor mean, a best-checkpoint restore, and reward shaping that encourages staying upright with small actions. I ran 30 independent seeds with 1200 episodes each.

Key Results & Analysis

The 30-seed sweep gave a mean final return of -1210 with a standard deviation of 363. Out of 30 seeds, only 4 reached a return better than -500, and most plateaued around -1400. This shows the method works but only some of the time, which matches what Sutton and Barto describe in §13.5 about the high variance of one-step actor-critic. The wide 95% CI in Figure 1 mostly comes from this split between successful and failed seeds.

The successful seeds learned a static balance rather than active stabilization. In Figure 3, episodes 0 and 1 settle at $\theta \approx -0.25$ and $\dot{\theta} \approx 0$ with a constant torque around $+0.6$. This makes physical sense because at that angle gravity needs $+0.6$ torque to cancel out, so the agent learned to provide exactly that. Interestingly, in an earlier experiment the agent did oscillate around $\theta = 0$ with small corrective torques, which was the active stabilization I originally expected. After I added reward shaping that rewards being upright with small actions, the agent shifted to the simpler static-tilt solution. The policy heatmap in Figure 5 shows why this works. The agent outputs positive torque almost everywhere with a narrow negative-torque region near upright, so once the pendulum balances it sits inside the positive region and stays there.

A few hyperparameters made a big difference. The actor mean needed a tanh squash to keep it inside the action range. Without it, $\mu(s)$ drifted outside ± 2 and every action ended up clipped to either -2 or $+2$. The standard deviation also needed a clamp, because the actor gradient kept pushing σ upward no matter what, which is why the entropy curve in Figure 2 is flat at the clamp value. Gradient clipping at 1.0 was enough to stop late-training collapse, but at 0.5 the actor barely learned at all. I also tried a reward shaping term that only penalized actions, but this made the agent settle at a tilted equilibrium with a large constant torque, since holding constant torque is cheaper than constantly correcting. After that I switched to penalizing tilt and rewarding being upright too. The TD-error spikes in Figure 4 are a good example of how unstable training is, since they show the critic having

a hard time keeping up with how fast the actor is changing. Overall, online TD(0) actor-critic was more fragile than I expected, and small things like the action range or σ growing unchecked can completely break training. This is probably why methods like PPO and SAC are more popular for this kind of problem.

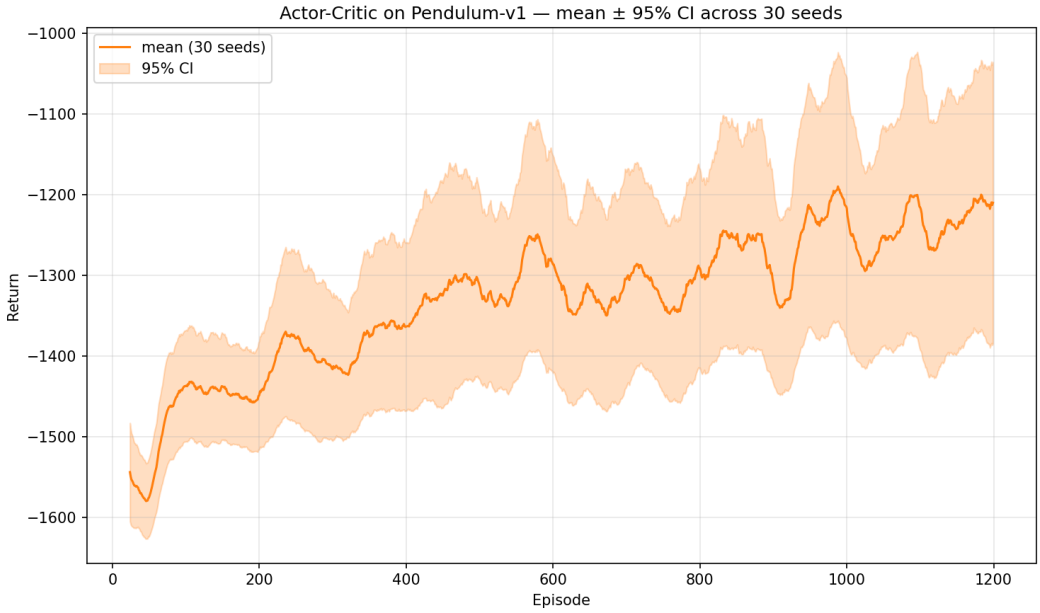

Figure 1: Mean return ± 95% CI across 30 independent seeds, smoothed with a 25-episode moving window. Each seed trains its own actor and critic from scratch with its own random initialization and environment reset seeds.

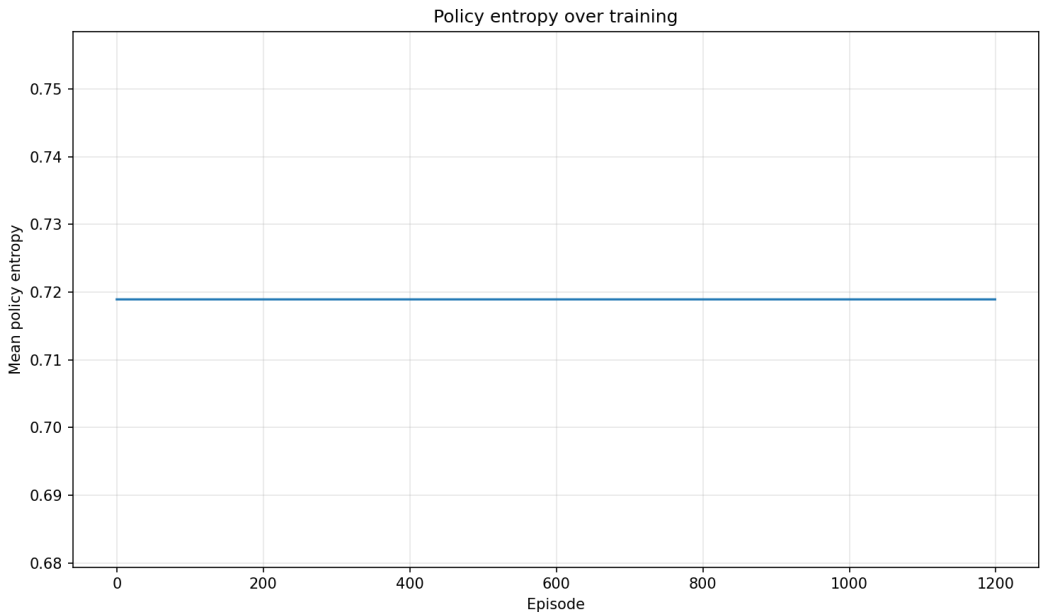

Figure 2: Mean policy entropy across training for a single seed (1200 episodes). Entropy is computed from σ of the Gaussian policy at each step and averaged per episode. Higher entropy means more exploration.

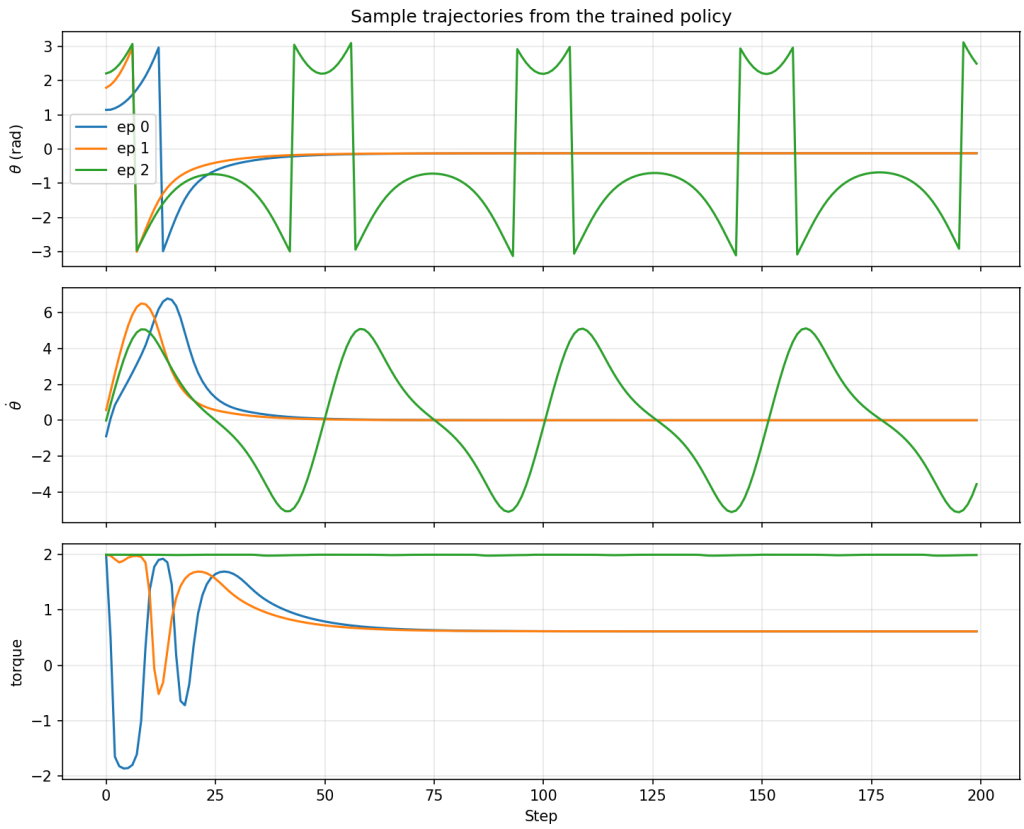

Figure 3: Three sample rollouts from the trained single-seed policy with reset seeds 123, 124, and 125. Top: pendulum angle θ in radians. Middle: angular velocity $\dot{\theta}$. Bottom: torque applied by the policy each step.

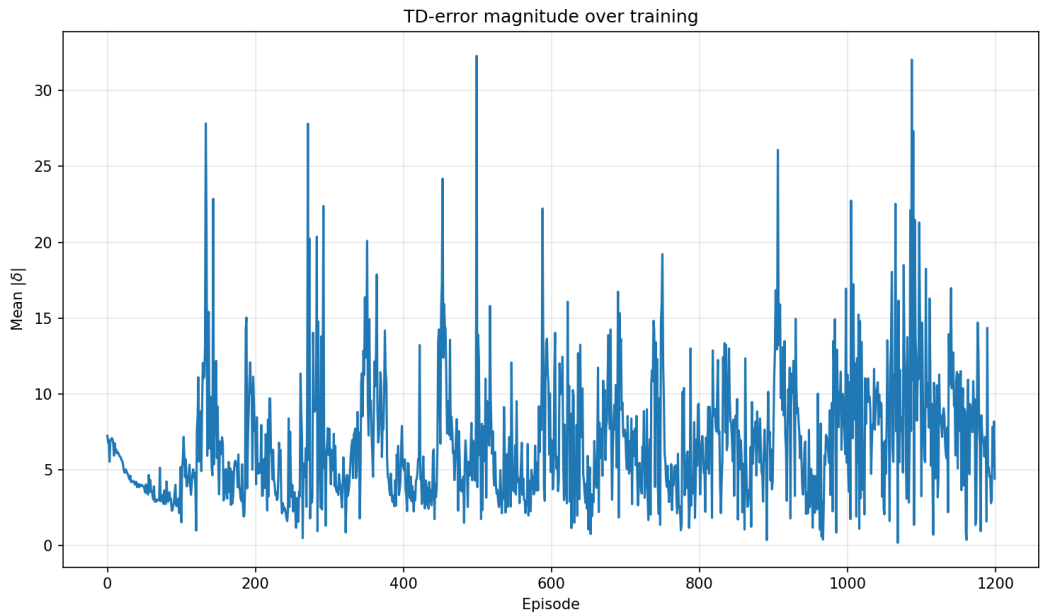

Figure 4: Mean $|\delta|$ per episode, where $\delta = r + \gamma V(s') - V(s)$ is the TD error used to update both networks. The critic minimizes δ^2 directly, and the actor uses δ as the advantage in its policy gradient update.

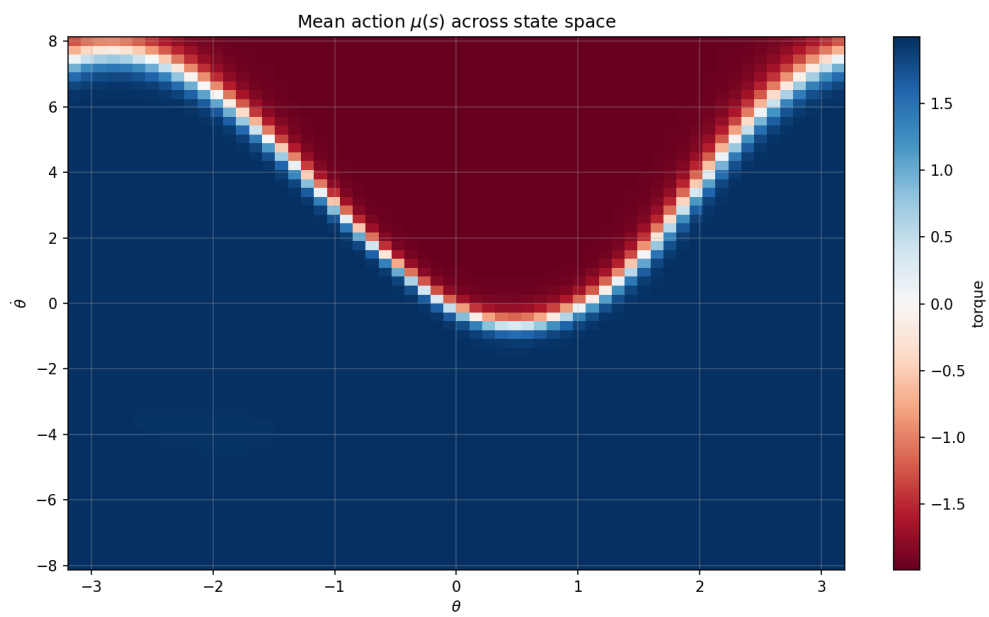

Figure 5: Mean action $\mu(s)$ of the trained policy across a 2-D slice of the state space, with θ on the x-axis and $\dot{\theta}$ on the y-axis. The 3-D state is reconstructed as $(\cos\theta, \sin\theta, \dot{\theta})$ before passing through the actor network. Color shows the torque the policy would apply.

Section 3: AI Use Reflection

Initial Interaction

Like previous labs, I started by talking through the week's concepts in ChatGPT. I had it review policy gradients and actor-critic methods from Sutton and Barto Chapter 13, then quiz me on the difference between value-based and policy-based methods. Once I felt comfortable with the concepts, I switched to Claude Code in Visual Studio Code. I asked Claude to review the lab directions and build a notebook template with markdown cells for each section before writing any code.

Iteration Cycle

Iteration 1: Actor mean saturated outside the action range

After the first training run, I looked at the policy heatmap and saw the colorbar running ± 15 even though Pendulum's action space is ± 2. The actor was outputting μ values way outside the legal range, and torch.clamp was doing all the work. I worked with Claude to add a tanh squash on the mean output so it stays inside the action range. After this fix the heatmap looked clean and training actually started learning a real policy.

Iteration 2: Standard deviation grew without bound

The next training run had log_std drift up to about $+2.0$, which means $\sigma \approx 7.85$. This was way bigger than the action range, so sampled actions were basically noise that got clipped. Claude explained that the actor gradient consistently pushes σ upward when advantages are noisy, so I added a clamp on log_std to keep σ in a reasonable range. This kept training stable for the rest of the experiments.

Iteration 3: Reward shaping pushed the agent to a lazy solution

I added reward shaping to encourage smaller actions when the pendulum was balanced. This made the agent settle at a tilted equilibrium with a constant torque around -1.25 instead of actively stabilizing at $\theta = 0$. After working through the math with Claude, I realized that a slight tilt with constant torque is cheaper than constantly correcting, so the agent took the easy way out. I switched the shaping to penalize tilt as well as actions, which moved the agent closer to upright.

Critical Evaluation

Compared to previous labs, Claude made more small mistakes during this one because actor-critic has many ways to fail. Most issues came down to PyTorch interactions like the action range mismatch and σ growing without bound, which were not obvious until I looked at the figures. I had to check each fix against Sutton and Barto, but Claude was helpful for explaining the gradient behavior at each step.

Learning Reflection

The main lesson from this lab is that policy gradient methods are much more sensitive to setup than value-based methods. With SARSA or Q-learning, even rough hyperparameters give reasonable results. With actor-critic, things like the action range, the σ clamp, and reward shaping completely change what the agent learns. Visualizations were critical for catching these issues since the failure modes were not obvious from the loss curves alone.

Section 4: Speaker Notes

- **Problem:** [FILL IN: 1 sentence problem statement and motivation]
- **Method:** [FILL IN: algorithm + key algorithmic choices, e.g. REINFORCE w/ baseline using a separate value network, Adam, $\gamma = 0.99$]
- **Design choice:** [FILL IN: an important decision you made — baseline architecture, initial $\log \sigma$, action clipping strategy, entropy bonus]
- **Key result:** [FILL IN: main quantitative finding, e.g. "baseline cut return variance by X%"]
- **Insight:** [FILL IN: what this taught you about policy gradients]
- **Challenge:** [FILL IN: hardest bug or design issue you faced]
- **Connection:** [FILL IN: how this builds on Lab 5 (function approx) and points toward DQN/PPO]

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*.
3. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
4. OpenAI. (2025). ChatGPT [Large language model]. `https://chat.openai.com`